

CSCI 611 Final Project Report:

Training A Graph Neural Network For Botnet Detection

Project Group 1: Logan M Flansaas and Montana B Pawek
Repository Link: https://github.com/MPawek/CSCI611_Group1

Introduction

A botnet is a coordinated network of compromised internet-connected devices. These devices can include computers, servers, routers, cameras, IoT devices, and other systems that have been infected and placed under attacker control. Once compromised, the devices can be used together to carry out automated attacks at large scale. Botnets are especially relevant to security because they turn ordinary devices into distributed attack infrastructure. A single infected device may not be powerful by itself, but thousands or millions of infected devices acting together can overwhelm services, spread malware, send spam, perform credential attacks, or support larger intrusion campaigns.

Botnets are difficult to defend against because they are distributed, scalable, quiet, and adaptable. There is often no single machine or address that defenders can block to stop the whole attack. Many botnets can operate silently for long periods before launching obvious attacks, and modern botnets may use low-rate or staggered communication that blends into normal traffic. This makes early detection difficult because defenders often do not directly observe the compromise itself. Instead, they have to infer malicious behavior from side effects such as who communicated with whom, how often they communicated, which protocols were used, and whether groups of hosts are behaving in coordinated ways.

The real-world impact of botnets is significant. One of their most visible uses is distributed denial-of-service attacks, where large numbers of compromised devices are used to overwhelm a target with traffic. In 2025, Cloudflare reported that the Aisuru botnet had an estimated 1–4 million infected hosts globally and routinely launched attacks exceeding 1 Tbps and 1 Bpps. Qrator Labs also reported mitigating a large L7 DDoS attack in September 2025 in which 5.76 million IP addresses were blocked during the incident. These examples show that botnets can affect governments, businesses, online services, and ordinary users by disrupting availability at massive scale.

Our project explores whether graph-based machine learning can help detect malicious hosts in a network earlier and more effectively than methods that treat network traffic as isolated events. Malicious behavior is not always obvious from a single device or a single traffic record. Instead, botnet behavior often becomes clearer when we look at patterns of communication across multiple hosts.

Existing botnet detection work includes traditional intrusion detection systems, standard machine learning models, and newer graph-based approaches. Traditional intrusion detection systems often rely on signatures or anomaly rules, which can work well for known threats but may struggle when attackers change behavior or hide coordination patterns. Standard machine learning methods such as Random Forest, SVM, Logistic Regression, and KNN can learn from engineered traffic features, but they often treat samples independently and may miss relationships among hosts. Graph-based methods are especially relevant to our project because they are designed for problems where relationships between entities matter, which fits the coordinated nature of botnets.

Our project matters because botnet detection is fundamentally a security problem about coordination, not just isolated malicious events. If defenders can identify suspicious communication patterns earlier, they may be able to contain infected devices before the botnet grows, launches attacks, or causes broader damage. The goal of this project is therefore to evaluate whether a graph-based machine learning approach is a useful direction for detecting botnet behavior and improving early network defense.

Our Solution

Our solution is an end-to-end graph learning framework for botnet detection. We start from network traffic data, transform it into host-to-host communication graphs, and train a Graph Neural Network to classify hosts as part of a botnet or not. The key design choice is that we preserve the structure of network communication instead of flattening the data into independent records. In our framework, each host is evaluated using both its own behavior and its relationships to other hosts in the network.

The uniqueness of our solution comes from matching the machine learning representation to the security problem. Botnet behavior often appears through coordination, repeated communication patterns, and relationships between infected devices and command-and-control infrastructure. A graph-based model gives us a way to encode those relationships directly. This allows the model to learn from local network neighborhoods rather than only from isolated traffic features, which is the main limitation of many traditional machine learning approaches.

We also organize communication into time-based graph snapshots so the model can analyze network behavior as it develops rather than as one large static graph. This makes the framework more practical for early detection because suspicious coordination may appear only during specific time periods. Overall, our solution is designed to test whether preserving network structure gives us a stronger signal for botnet detection than methods that ignore how hosts are connected.

Implementation

To evaluate the graph learning framework, we implemented a complete pipeline for graph-based botnet detection using Python, PyTorch, and PyTorch Geometric. The system begins with the network flow data processing.

We preprocessed the CTU-13 NetFlow data by converting raw flow records into 5 minute time-windowed communication graphs. Each .binetflow file was streamed row by row, grouped into fixed-size time windows, and accumulated into one graph per window. In each graph, we treated each IP address active in that window as a node, and each source-to-destination communication pair as a directed edge. This gave the GNN the structure it needs: not just traffic records, but a graph showing which hosts communicated with which other hosts. Our final dataset contained 1,594 graph windows, with approximately 7.5 million total nodes and 7.9 million total edges. On average, each window contained about 4,734 nodes and 4,962 edges, with a graph size standard deviation of roughly 5,000 nodes.

For each node, we built a 24-dimensional node feature vector describing host-level behavior during that window. These features included in-degree, out-degree, incoming and outgoing flow counts, packet totals, byte totals, source-byte totals, unique peers, unique ports, mean duration, and incoming/outgoing protocol counts for TCP, UDP, ICMP, and other protocols. We also log-scaled the numeric feature values using $\log_{1p}$ so that very large byte, packet, or flow counts would not dominate the feature space.

For each edge, we built an 11-dimensional edge feature vector describing communication-level behavior between a source host and destination host. These edge features included number of flows, total packets, total bytes, source bytes, mean duration, unique source ports, unique destination ports, and TCP/UDP/ICMP/other protocol counts. This let us preserve both the topology of the communication graph and the behavior of each connection.

We labeled nodes using a hybrid strategy. When scenario README labels were available, we used the known infected and normal host IPs; otherwise, we fell back to flow labels such as from-botnet and from-normal. Each graph stored labels as `y`, with 1 for botnet/infected nodes, 0 for normal nodes, and -1 for ignored or unlabeled nodes. We also stored a `labeled_mask` so training and evaluation only used nodes with valid labels. This was important because only about 0.13% of labeled nodes were positive, so class imbalance was a major issue in the project.

Finally, we converted the saved JSON graph snapshots into PyTorch Geometric Data objects. Each trainable graph contained `x` for node features, `edge_index` for the directed graph structure, `edge_attr` for edge features, `y` for node labels, and `labeled_mask` for selecting valid training labels. We also preserved metadata such as node IPs, feature names, scenario name,

source file, window start/end time, original node count, and original edge count. This conversion step produced the graph format our GNN could train on directly.

Using these 5-minute graph snapshots, we trained and evaluated several Graph Neural Network architectures for binary node classification, where the goal was to determine whether a node was associated with botnet activity. These variants included a baseline Graph Convolutional Network (GCN), a threshold-tuned version of the same model, an edge-aware GNN, and deeper GNN configurations. Experiments were evaluated using a train, validation, and test graphs, and the data was split by scenario, not by graph. Of the roughly 1500 total graphs divided amongst scenarios, there ended up being ~1300 graphs for training, 5 for validation, and 250 for testing.

All models were trained using the Adam optimizer along with individually tuned learning rates as well as L2 weight decay regularization. Cross-entropy loss was the function chosen, and class weighting was used to fix the imbalance between the benign and malicious nodes. Training was performed across all graph snapshots contained in the training split. During each training epoch, the graph snapshots were loaded sequentially, the node predictions were computed, and the loss was evaluated on the supervised nodes. The gradients were then backpropagated, and the model parameters were updated, in the same manner used in most other neural networks. Validation and testing were performed on separate graph splits to evaluate generalization to previously unseen network snapshots. Training used validation-based early stopping to reduce overfitting, meaning the model state that achieved the highest validation F1 score was saved and restored prior to the final test evaluation.

```

# Initial GNN model with no added features
class SimpleGCN(nn.Module):
    def __init__(
        self,
        in_channels: int,
        hidden_dim: int,
        out_channels: int,
        dropout: float = 0.3,
    ):
        super().__init__()
        # Dropout for regularization
        self.dropout = dropout

        # Conv layers similar to a CNN
        self.conv1 = GCNConv(in_channels, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, hidden_dim)
        self.classifier = nn.Linear(hidden_dim, out_channels)

    def forward(
        self,
        x: torch.Tensor,
        edge_index: torch.Tensor,
    ) -> torch.Tensor:
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)

        logits = self.classifier(x)
        return logits

```

Figure 1: The simple GCN architecture

The baseline GCN was implemented using PyTorch Geometric's GCNConv layers. As can be seen in Figure 1, the architecture consisted first of a graph convolution layer, followed by ReLU activation dropout regularization, a second convolution layer, and a final linear classification layer. This baseline GCN used normalized neighborhood aggregation to update node representations. At each layer, the node embeddings were updated using information from neighboring nodes connected through the graph structure. This model only used node features and graph topology to make classifications, and did not incorporate edge attributes or temporal relationships in its determination. As there were 24 node features per node, the input dimension was 24. The hidden dimension used the standard initial GCN value of 32, resulting in a model structure of 24 -> 32 -> 32 -> 2.

```

probs = torch.softmax(logits, dim=1)[: , 1]
preds = (probs >= THRESHOLD).long()

```

Figure 2: The two-line addition of code that added the threshold feature

Initial experimentation with this baseline showed that the default argmax classification GCN examples used produced predictions that were too conservative, which resulted in a lower recall. After baseline testing, the hyperparameters were tuned using validation to find the optimal configuration. This resulted in another imbalanced precision to recall ratio, indicating that something would have to be done. To improve the balance between precision and recall, the next version of the model implemented probability thresholding, as shown in Figure 2. With this, instead of selecting the class with the highest logit value, softmax probabilities for the botnet class were computed, and predictions were made using a configurable threshold. This threshold value was determined using validation-set performance.

Parameter	Hidden Dimensions	Learning Rate	Weight Decay	Dropout	Patience	Threshold	Optimizer	Loss
Final Value	32	0.01	1e-6	0.3	50	0.6	Adam	Weighted Cross-Entropy

Figure 3: Final hyperparameter values determined through tuning

To evaluate whether the addition of edge features would improve detection performance, an edge-aware Graph Attention Network architecture was implemented. This model specifically incorporated edge attributes into message-passing operations and attention score computation. In this way, message construction used the source node embeddings, the destination node embeddings, as well as the edge feature embeddings. There were a total of 11 edge features per edge, including statistics such as the number of flows between the two hosts, the total packet count, the total bytes transferred, protocol statistics, and number of ports used. This surprisingly resulted in lower overall F1 performance than the simpler baseline GCN during experimentation. This lowering of performance suggests that perhaps introducing edge features resulted in too much additional noise under the architecture's current configuration.

Another direction the architecture could be taken in was to add additional layers to the model. These deeper layers made a node's neighborhood larger to include nodes two connections away from the target node. Due to the way GNNs work, the additional layer did not have an increase in size, remaining a 32-input layer. After adding a single additional layer to test this architecture, the tests showed limited improvement and actually showed signs of

over-smoothing; the node embeddings shared too much information and became increasingly similar, which impacted the model’s ability to determine class accurately. The two-layer GCN with the performance threshold remained the most stable and effective configuration across experiments.

Each model’s performance was evaluated using precision, recall, and an F1 score. Additionally, the PR-curve was used to determine the optimal threshold value for the second model onwards. Due to the fact that botnet detection is an imbalance classification problem, the F1 score served as the primary evaluation metric. The final results were averaged across multiple random seeds to improve robustness and reduce variance.

Results and Analysis

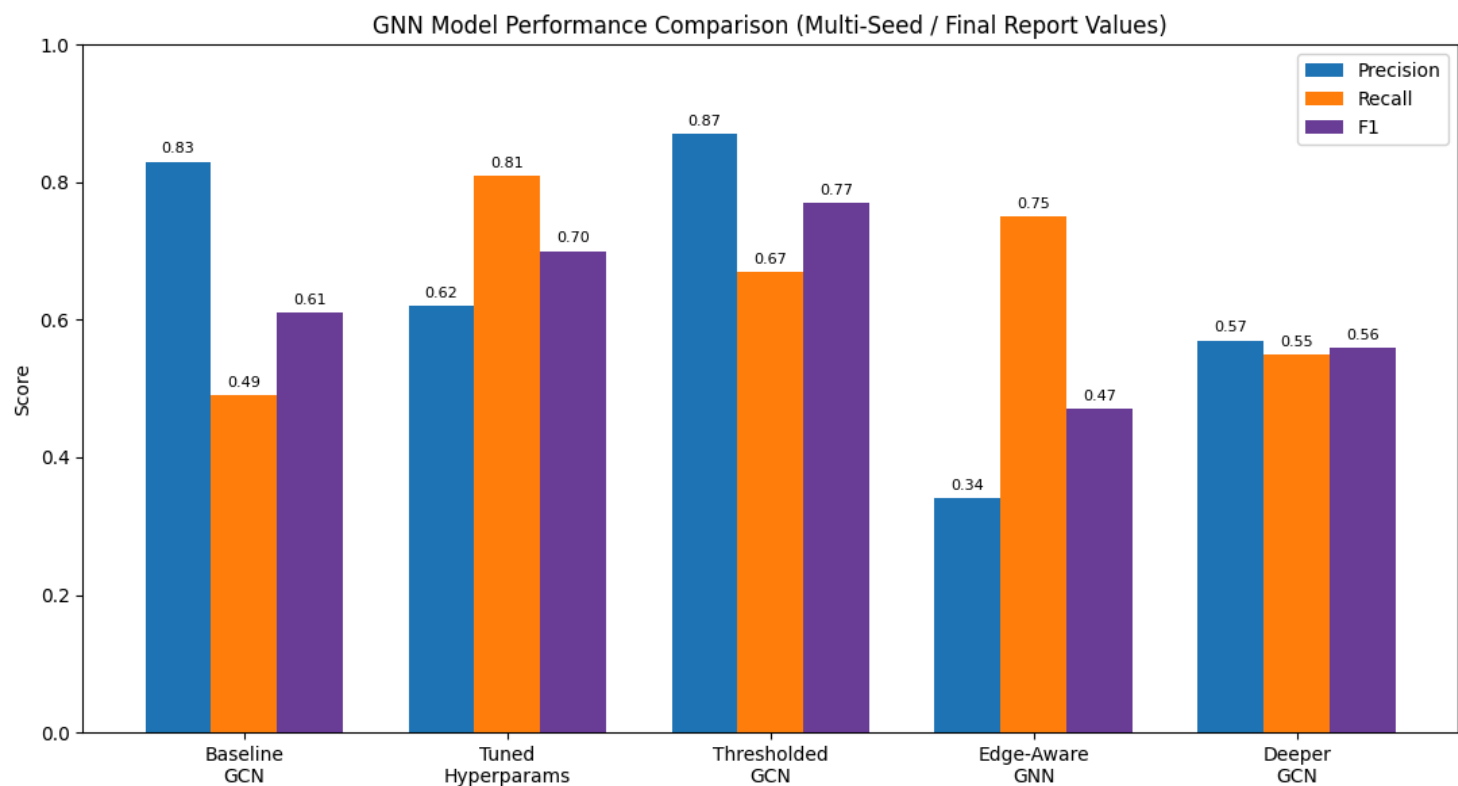


Figure 4: Multi-seed precision, recall, and F1 performance across models

The initial baseline model only used node features and graph connectivity information. Initial experiments demonstrated that the graph-based representation did meaningfully capture malicious botnet behavior patterns. Even with no additional methods or architecture, the baseline GCN achieved F1 scores that averaged around 0.61, showing that this method carried

meaningful detection signals. This score indicates that using a graph to represent hosts and connections did provide some useful signals for botnet detection. These early experiments also showed a strong imbalance between precision and recall – precision was tested at 0.83 and recall at 0.49. The baseline model was conservative, and only identified infected hosts when it was certain they were infected. Other hosts may have been suspected to be infected, but the model was not confident enough to classify them as such. Hyperparameter tuning was the next obvious step to improve the model's performance.

To improve performance, several training hyperparameters were explored, including the learning rate, the hidden dimension size, the dropout rate, the weight decay, and the patience (early stopping threshold). This tuning did improve convergence stability and overall model performance. After testing at various magnitudes, it was determined that a learning rate of 0.01 performed the best during validation, despite being larger than what is traditionally suggested for GNNs. The other major adjustment was lowering the weight decay to 0.000001; this solved some of the underfitting the model was previously doing due to the data imbalance. This tuning resulted in an F1 score of ~ 0.70 , a precision of 0.62, and a recall of 0.81, which were much improved over the baseline metrics. The magnitude of this improvement showed that the GCN was sensitive to optimization settings. The model still exhibited unbalanced precision and recall, outputting a large number of false positives. This indicated that we would need to address this imbalance to further improve the model, as we wanted to make sure precision was the main focus while not allowing recall to get too low.

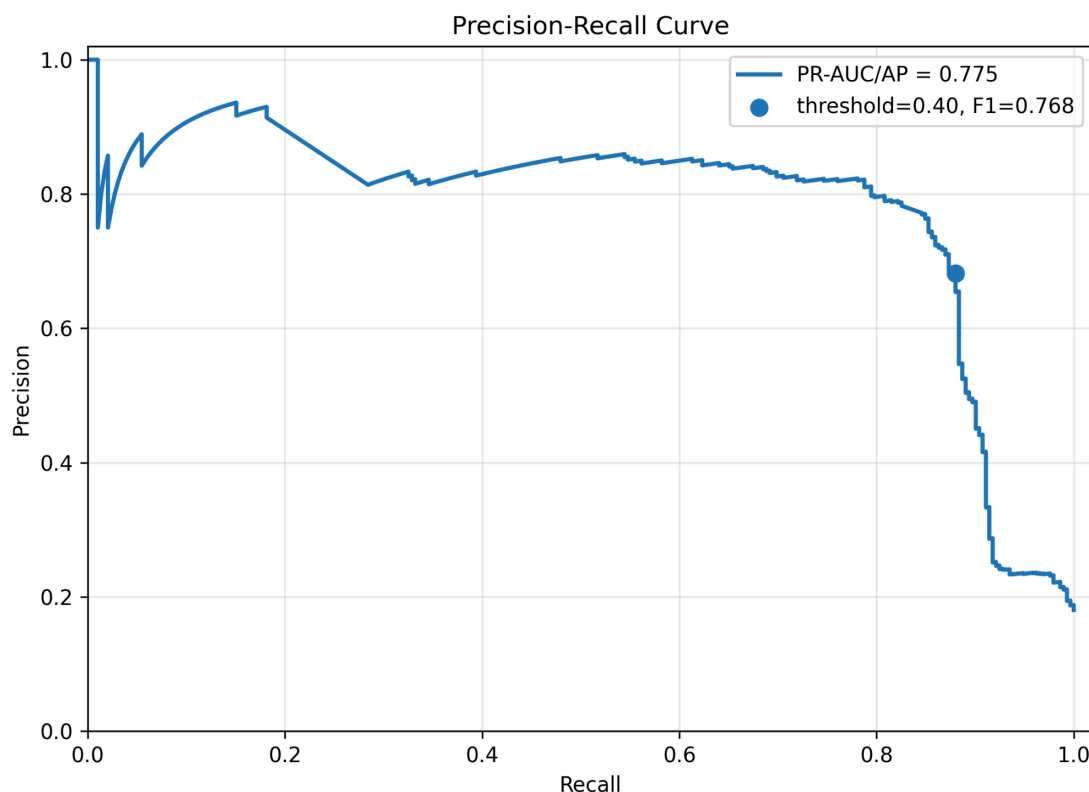


Figure 5: Using the PR Curve to determine the optimal threshold value. (The plotted threshold corresponds to the opposite class probability, so the implemented malicious-class threshold was calculated as $1 - t$.)

The largest performance improvement occurred through probability threshold tuning. Initially, the class predictions were generated using the default argmax classification strategy; the problem with this is that since our data was so imbalanced, the decision boundaries were skewed. While recall is still important to reduce the amount of false positive alerts, precision is even more important to actually detect the botnet. To address this imbalance, the classification thresholds were adjusted using validation-set probability outputs and the PR-curve, as seen in Figure 5. Lowering the threshold improved recall but lowered precision, while increasing the threshold did the opposite. The best overall balance was achieved using a threshold of approximately 0.6, resulting in an increase of the F1 score to 0.77, and precision and recall to 0.87 and 0.67, respectively – the strongest configuration observed during the full course of the experiment. This increase showed that the baseline model was capable of identifying compromised nodes, but was not confident in classifying them as such initially likely due to the lack of examples. It's likely that additional training with more botnet examples could increase performance even further.

The edge-aware GNN was implemented to test whether incorporating communication-level edge attributes could improve performance beyond what the standard GCN

could achieve. After testing, however, it appeared that this version performed worse than the previous configurations – an F1 score of 0.47, precision of 0.34, and a recall of 0.75 suggest that the model was overly loose with predicting botnet activity. The edge-aware architecture seems to indicate that the increase of information made the model too sensitive, and it was not able to learn a clean distinction between benign and malicious connections. The explanation we found most likely is that the addition of 11 more features produced too much additional noise, especially in the context of those features applying only to that 5-minute window. Perhaps aggregated statistics for the entire segment of time would have been better for aiding detection. The additional features also made the model much more complex, making it harder to optimize compared with the simpler GNN. It is still our opinion that the addition of edge features should eventually yield better results than the initial architecture, but the specific edge features that provide the strongest signal need to be isolated first. It's also possible that combining this approach with a temporal aspect that is aware of changes in this data between time windows would be a more effective implementation of this idea.

While the baseline GCN used two graph convolution layers, allowing it to use information from a two-hop neighborhood, the deeper GNN added an additional graph convolution layer, which resulted in an 24 -> 32 -> 32 -> 32 -> 2 setup of feature dimensions. This allowed the model to use information from a three-hop neighborhood for a bigger picture of network activity. Despite this additional context information, the deeper GNN did not improve performance over the baseline or thresholded GCN. The deeper model produced an F1 score of 0.56, with precision and recall values around 0.57 and 0.55, respectively. This would suggest that the additional layer didn't help the model learn more useful information, and instead likely introduced more noise or confused class separability. Providing too much information may blur the distinction between benign and malicious nodes. A common limitation in deep GCN architectures is over-smoothing, where node embeddings can become more and more similar because each node repeatedly averages information from larger neighborhoods. If a wider neighborhood connects more nodes to suspicious botnet activity, even if most nodes are benign, they still have botnet signals in their aggregated data. Overall, the multi-layer experiment showed that simply increasing GNN depth did not automatically improve botnet detection performance, and that the initial 2-layer GCN baseline was still an effective configuration.

Overall, the strongest result from this project was that a relatively simple 2-layer GCN - when combined with careful threshold tuning - performed better than more complex architectural variants. The baseline also showed that preserving host-to-host communication in graph form provides a useful way to search for botnet signals using edges and nodes. Hyperparameter tuning increased stability and performance, but the main performance gain was from threshold tuning which increased precision without lowering recall much more effectively than everything else in the imbalanced scenarios. The more complex models showed much lower performance, likely

due to the addition of more noise in a situation that already resembled searching for a needle in a haystack.

In the context of work in this field, these findings follow a common pattern – graph-based models are promising tools to detect botnets due to the fact they preserve relationships that traditional methods often flatten away. However, it's important to note that Graph Neural Networks are very sensitive to representation design, calibration, and methodology. Even simple models require careful design and a large amount of tuning to work properly, and the more complex the model is, the more these requirements increase. At the end of the day, this project supports the value of graph learning for botnet detection while highlighting that performance gains in this field come just as much from calibration, threshold detection, and disciplined evaluation as implementing complex GNN architectures.

Conclusion/Future Work

In this project, we show that graph-based machine learning is a practical and promising direction for botnet detection applications. By converting NetFlow communications into time-windowed host-to-host communication graphs, we were able to train a GNN that learned from both host-level behavior and behavior of each host's neighbors and the surrounding network structure. Our strongest result came from the threshold tuned two layer GCN, which outperformed the more complex edge-aware and deeper 3 layer GNN models. This shows that preserving graph structure is useful, but that more complexity does not automatically improve detection. In this problem, careful graph design, feature design, class imbalance handling, calibration, and threshold selection are just as important as the model architecture.

For future work, we would first expand training across more botnet datasets and scenarios so that our model can better generalize beyond CTU-13. Since our positive class was very small, adding more examples would likely improve the robustness and accuracy of our models and reduce the model's dependence on threshold tuning. We would also revisit edge features using feature selection and experiment with longer or shorter window aggregation, since the edge aware model may have performed poorly because it introduced too much noisy information at the 5 minute window level. Another promising direction would be to model the graphs temporally instead of in windowed snapshots. A temporal GNN or Temporal Graph Network-style model could learn how host behavior changes across consecutive windows, which may help detect early coordination before a botnet becomes obvious and better detect brief spikes in network traffic that may be lost in the aggregation windows.

References

Cloudflare, *Cloudflare's 2025 Q3 DDoS Threat Report*.
<https://blog.cloudflare.com/ddos-threat-report-2025-q3/>

Fey, Matthias, and Jan Eric Lenssen. *Fast Graph Representation Learning with PyTorch Geometric*. ICLR Workshop, 2019.

Gilmer, Justin, et al. *Neural Message Passing for Quantum Chemistry*. ICML, 2017.

Jiawei Zhou, Zhiying Xu, Alexander M. Rush, and Minlan Yu, *Automating Botnet Detection with Graph Neural Networks*. <https://arxiv.org/abs/2003.06344>

Kipf, Thomas N., and Max Welling. *Semi-Supervised Classification with Graph Convolutional Networks*. ICLR, 2017.

Qrator Labs, *Massive L7 DDoS Botnet Expands to 5.76M Devices*.
https://blog.qrator.net/en/massive-l7-ddos-botnet-expands-to-576m-devices-qrator_218/

Stratosphere IPS, *The CTU-13 Dataset: A Labeled Dataset with Botnet, Normal and Background Traffic*. <https://www.stratosphereips.org/datasets-ctu13>

Sebastian Garcia, Martin Grill, Jan Stiborek, and Alejandro Zunino, *An Empirical Comparison of Botnet Detection Methods*,
https://www.researchgate.net/publication/262921640_An_Empirical_Comparison_of_Botnet_Detection_Methods

Sajjad Arshad, Maghsoud Abbaspour, Mehdi Kharrazi, and Hooman Sanatkar, *An Anomaly-Based Botnet Detection Approach for Identifying Stealthy Botnets*.
<https://arxiv.org/abs/1811.00925>

Veličković, Petar, et al. *Graph Attention Networks*. ICLR, 2018.